

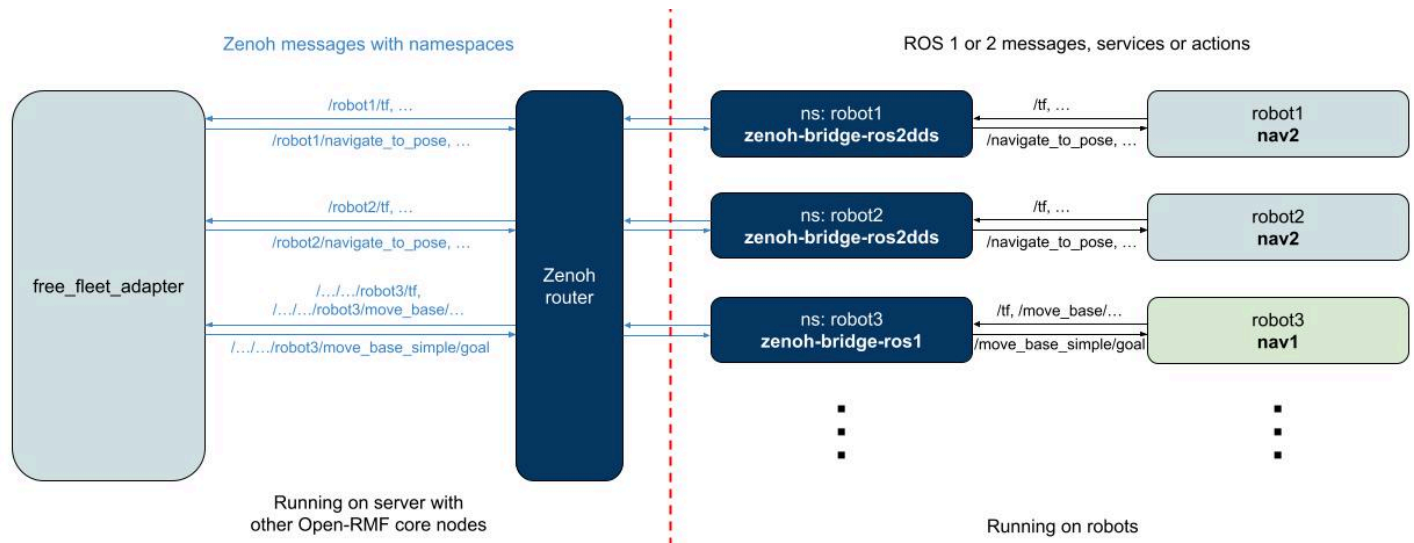
Free Fleet Adapter

The free fleet adapter formerly known as Free Fleet, allows users to integrate standalone mobile robots to form a heterogeneous fleet, to work with Open-RMF.

The `free_fleet_adapter` implements the Easy Full Control fleet adapter API, and communicates with individual robots over Zenoh bridges. Out-of-the-box, it supports robots running [ROS 1 Navigation](#) as well as [ROS 2 Navigation2](#).

Check out the [simulation examples](#) for more information.

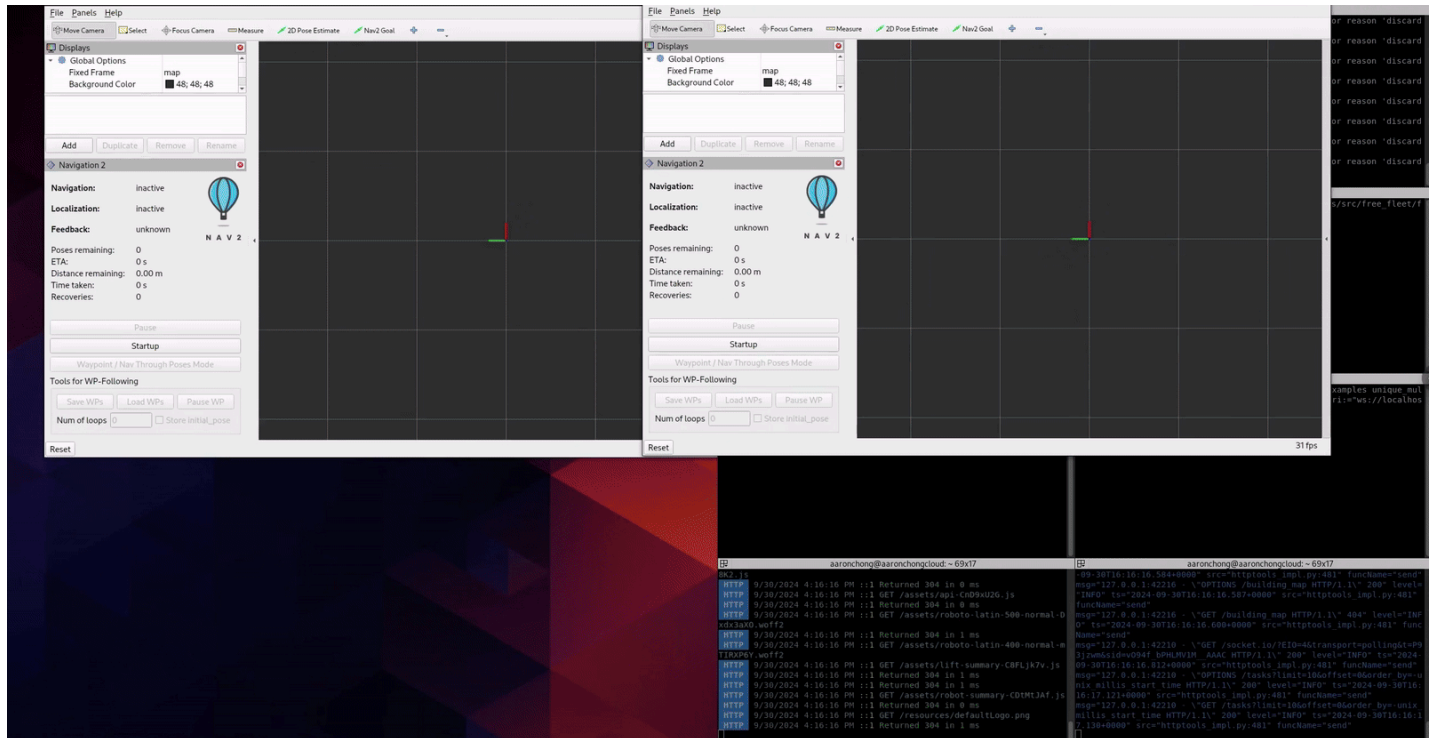
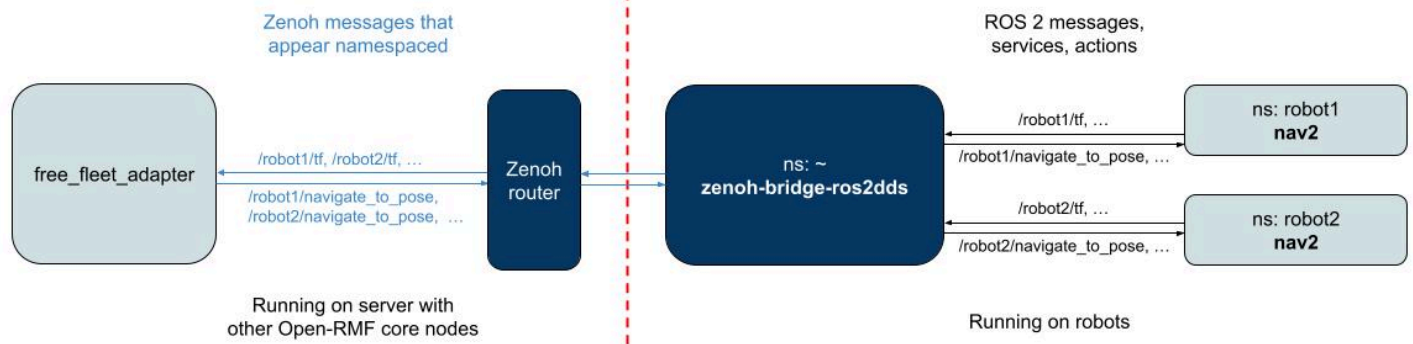
Below is the ideal integration architecture of several robots that are running either ROS 2 Navigation2 or ROS 1 Navigation.



With Zenoh bridges, users should be able to integrate their robot as-is, without additional namespacing. The Zenoh bridges are responsible for filtering topics to reduce network traffic, and to namespace the various Zenoh keys, which the `free_fleet_adapter` operates on. Handling message definitions across domains have been greatly simplified too, thanks to [pycdr2](#) for ROS 2 messages and [rosbags](#) for ROS 1 messages.

ROS 2 Navigation2

For ROS 2 Navigation2, the integration is enabled by [zenoh-bridge-ros2dds](#). With an appropriately configured bridge running on the robot, the `free_fleet_adapter` is able to communicate and control the robot, via its navigation behavior trees.



The Navigation2 integration currently uses the simple `navigate_to_pose` behavior to provide navigation commands from point to point following the RMF waypoints defined. Support for utilizing more Navigation2 behaviors or custom behaviors is on the roadmap, but in the meantime, contributions are always welcome!

Below is an example of the robot defined in the fleet configuration file, you can find the file [here](#). Take note of the defined `navigation_stack`, this allows the `free_fleet_adapter` to pick the `Nav2RobotAdapter` for this particular robot.

```

...
robots:
  nav2_tb3:
    charger: "tb3_charger"
    responsive_wait: False # Should responsive wait be on/off for this specific
robot? Overrides the fleet-wide setting.
    # For Nav2RobotAdapter
    navigation_stack: 2
    initial_map: "L1"
    maps:
      L1:
        map_url: "/opt/ros/jazzy/share/nav2_bringup/maps/tb3_sandbox.yaml"
...

```

Below is an example of the Zenoh configuration file used by the `zenoh-bridge-ros2dds` on the robot. The Zenoh keys are namespaced by the `namespace` parameter, while we are able to filter and rate-limit ROS 2 topics, services, actions to reduce network traffic. With the `client` mode, the bridge will seek to connect to a locally available Zenoh router.

```

{
  plugins: {
    ros2dds: {
      namespace: "/nav2_tb3",
      allow: {
        publishers: [".*/tf", ".*/battery_state"],
        subscribers: [],
        service_servers: [],
        service_clients: [],
        action_servers: [".*/navigate_to_pose"],
        action_clients: [],
      },
      pub_max_frequencies: [".*/navigate_to_pose/*=1", ".*/battery_state=1"],
    },
  },
  // Zenoh related configurations
  mode: "client",
  connect: {
    endpoints: []
  },
  listen: {
    endpoints: []
  },
}

```

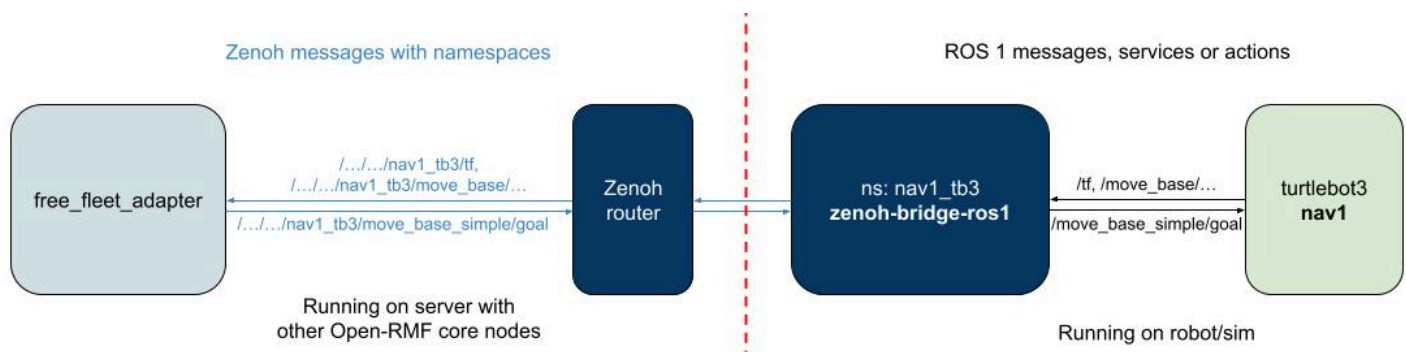
Check out our [integration test setup](#) to get a better idea on how this integration can be deployed.

ROS 1 Navigation Stack

i The zenoh-bridge-ros1 is an experimental bridge, and is actively being iterated on. Especially with [ROS 1 going EOL in May 2025](#), users are encouraged to migrate to ROS 2.

The ROS 1 Navigation integration has only been tested in simulation and in ROS 1 Noetic, and is currently still using a fork of [zenoh-plugin-ros1](#), to support bridge namespacing. This will be updated after contributions to upstream has been made.

For ROS 1 Navigation, the integration is enabled by [zenoh-bridge-ros1](#). Similarly, with an appropriately configured bridge running on the robot, the `free_fleet_adapter` is able to communicate and control the robot, via the `move_base` topics.



Below is an example of the robot defined in the fleet configuration file, you can find the file [here](#). Take note of the defined `navigation_stack`, this allows the `free_fleet_adapter` to pick the `Nav1RobotAdapter` for this particular robot.

```
...
robots:
  nav1_tb3:
    charger: "tb3_charger"
    responsive_wait: False # Should responsive wait be on/off for this specific
    robot? Overrides the fleet-wide setting.
    # For Nav1RobotAdapter
    navigation_stack: 1
    initial_map: "L1"
    maps:
      L1:
        # The nav1 robot will generally no access to this nav2_bringup package,
        this should point to where the nav1 map is stored.
        map_url: "/tmp/navigation2/nav2_bringup/maps/tb3_sandbox.yaml"
    ,,,
```

Below is an example of the Zenoh configuration file used by the `zenoh-bridge-ros1` on the robot. The Zenoh keys are namespaced by the `bridge_namespace` parameter, while we are

able to filter ROS 1 topics and services to reduce network traffic. With the `client` mode, the bridge will seek to connect to a locally available Zenoh router.

```
{
  plugins: {
    ros1: {
      bridge_namespace: "nav1_tb3",
      subscriber_bridging_mode: "disabled",
      publisher_bridging_mode: "disabled",
      service_bridging_mode: "disabled",
      client_bridging_mode: "disabled",
      // Zenoh -> ROS 1
      subscriber_topic_custom_bridging_mode: {
        "/move_base_simple/goal": "auto",
        "/move_base/cancel": "auto",
      },
      // ROS 1 -> Zenoh
      publisher_topic_custom_bridging_mode: {
        "/tf": "auto",
        "/battery_state": "auto",
        "/move_base/status": "auto"
      },
    },
  },
  // Zenoh related configurations
  mode: "client",
  connect: {
    endpoints: []
  },
  listen: {
    endpoints: []
  },
}
```

Check out our [integration test setup](#) to get a better idea on how this integration can be deployed.

Custom Navigation Stack

Users looking to integrate their own custom navigation stack, can implement the abstract interfaces in `RobotAdapter` [here](#).

The implementation will need a name, for the `free_fleet_adapter` to correctly allocate adapters to each connected robot. This name is configured in the robot's configuration in the fleet configuration file. For example, Navigation2 is just `2`, while ROS 1 Navigation is `1`.

Contributions are always welcome!

More information and links

- [Repository](#)
- Free fleet [legacy implementation](#) and [documentation page](#).
- [zenoh-bridge-ros2dds](#)
- [Custom zenoh-bridge-ros1 fork](#)

Found a bug? [Edit this page on GitHub](#).

Legacy - Free Fleet

 This legacy implementation is no longer being supported, check out the most up-to-date implementation [here](#).

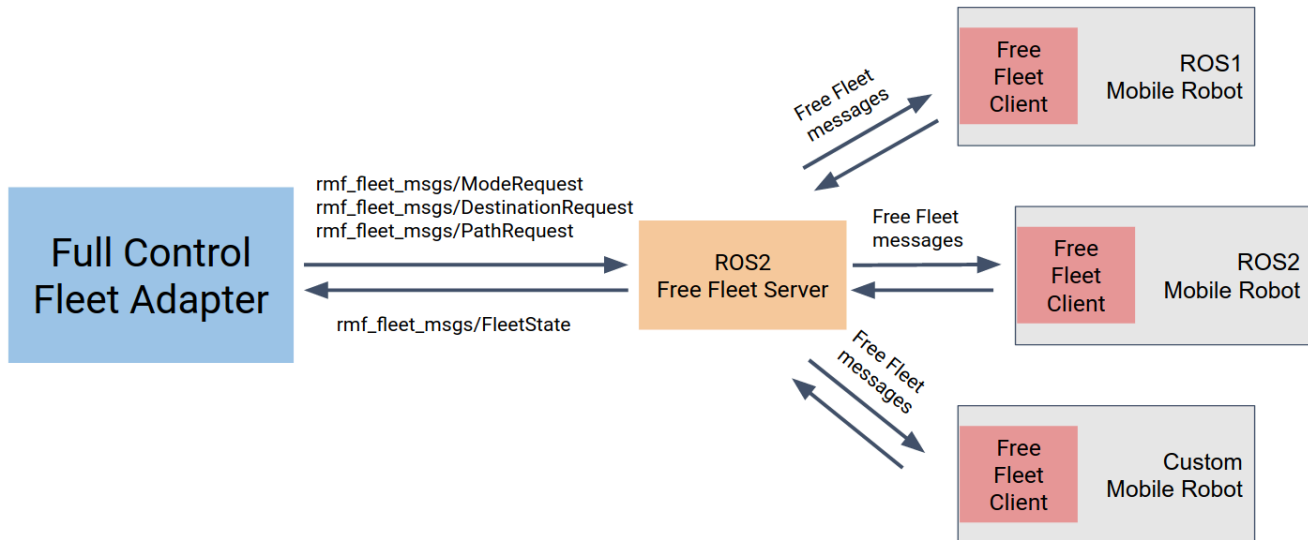
In the event that the user wishes to integrate a standalone mobile robot which doesn't come with its own fleet management system, the open source fleet management system `free_fleet` could be used.

The `free_fleet` system is split into a client and a server. The client is to be run on each of these standalone mobile robots alongside their navigation software and is intended to have direct control over the mobile robot while at the same time monitor its status and report back to the server. The client's base implementation is designed to allow interaction with different configurations of mobile robots yet report to the same server. This way, users are able to use `free_fleet` to manage a heterogeneous fleet of robots, each using different distributions of ROS, versions of ROS, navigation software, or onboard communication protocols.

The server is run on a central computer and consolidates the incoming status updates from each client to be either visualized using a UI, or relayed upstream to RMF. The server also relays commands from the user via the UI or from RMF down to the clients to be executed. Each server can work with multiple clients at a time, hence it serves the role of a fleet management system. The server can be implemented and used as its own fleet management system or work with larger systems like RMF, bridging the gap between each mobile robot's API and RMF's API and interface.

The communication between the `free_fleet` server and `free_fleet` clients is implemented using `CycloneDDS`, therefore we are not concerned if the mobile robot or central computer is running different versions of ROS.

In this section, we will address 4 different approaches of using `free_fleet` to integrate with RMF, specifically the navigation stack used by the robot. Each approach maintains a similar systems architecture, which is illustrated in the simple block diagram below, but there are specific examples depending on the software choice for the navigation stack used by the robot developer.



ROS 1 Navigation Stack

An implementation of a `free_fleet` client that works with a ROS 1 navigation stack can be found in the [free_fleet repository](#). The implementation expects the transforms of the mobile robot to be fully defined, the mobile robot to accept navigation commands via the `move_base` action library, as well as publishing its battery status published using the `sensor_msgs/BatteryState` message.

After following the build instructions on the README on the mobile robot, the user can launch the client as part of their launch script while at the same time define the necessary parameters using `rosparam`. Below is a small snippet example of how a client can be launched, with its parameters defined,


```

<node name="free_fleet_client_node"
  pkg="free_fleet_client_ros1"
  type="free_fleet_client_ros1" output="screen">

  <!-- These parameters will be used to identify the mobile robots -->
  <param name="fleet_name" type="string" value="example_fleet"/>
  <param name="robot_name" type="string" value="example_bot"/>
  <param name="robot_model" type="string" value="Turtlebot3"/>

  <!-- These are the topics required to get battery and level information -->
  <param name="battery_state_topic" type="string"
value="example_bot/battery_state"/>
  <param name="level_name_topic" type="string" value="example_bot/level_name"/>

  <!-- These frames will be used to update the mobile robot's location -->
  <param name="map_frame" type="string" value="example_bot/map"/>
  <param name="robot_frame" type="string" value="example_bot/base_footprint"/>

  <!-- The name of the move_base server for actions -->
  <param name="move_base_server_name" type="string"
value="example_bot/move_base"/>

  <!-- These are DDS configurations used between Free Fleet clients and servers --
>
  <param name="dds_domain" type="int" value="42"/>
  <param name="dds_state_topic" type="string" value="robot_state"/>
  <param name="dds_mode_request_topic" type="string" value="mode_request"/>
  <param name="dds_path_request_topic" type="string" value="path_request"/>
  <param name="dds_destination_request_topic" type="string"
value="destination_request"/>

  <!-- This decides how long the client should wait for a valid transform and
action server before failing -->
  <param name="wait_timeout" type="double" value="10"/>

  <!-- These define the frequency at which the client checks for commands and
publishes the robot state to the server -->
  <param name="update_frequency" type="double" value="10.0"/>
  <param name="publish_frequency" type="double" value="1.0"/>

  <!-- The client will only pass on navigation commands if the destination or
first waypoint
of the path is within this distance away, otherwise it will ignore the command -
->
  <param name="max_dist_to_first_waypoint" type="double" value="10.0"/>

</node>

```

The running `free_fleet` client will communicate with the nodes running on the robot via ROS 1, while publishing its state and subscribing to requests over DDS with the `free_fleet` server.

The current implementation of the `free_fleet` server is implemented with ROS 2 and communicates with RMF using the aforementioned ROS 2 message and topic interfaces of an RMF fleet adapter. The ROS 2 build instructions can also be found on the same repository. Similar to the client, a simple ROS 2 wrapper has been implemented, and it can be started using a `.launch.xml` file like so:

```
<node pkg="free_fleet_server_ros2"
  exec="free_fleet_server_ros2"
  name="free_fleet_server_node"
  node-name="free_fleet_server_node"
  output="both">

  <!-- Fleet name will be used to identify robots -->
  <param name="fleet_name" value="example_fleet"/>

  <!-- These are the ROS2 topic names that will be used to communicate with RMF -->
  <param name="fleet_state_topic" value="fleet_states"/>
  <param name="mode_request_topic" value="robot_mode_requests"/>
  <param name="path_request_topic" value="robot_path_requests"/>
  <param name="destination_request_topic" value="robot_destination_requests"/>

  <!-- These are the DDS specific configurations used to communicate with the
  clients -->
  <param name="dds_domain" value="42"/>
  <param name="dds_robot_state_topic" value="robot_state"/>
  <param name="dds_mode_request_topic" value="mode_request"/>
  <param name="dds_path_request_topic" value="path_request"/>
  <param name="dds_destination_request_topic" value="destination_request"/>

  <!-- This determines the frequency it checks for incoming state and request
  messages,
  as well as how often it publishes its fleet state to RMF -->
  <param name="update_state_frequency" value="20.0"/>
  <param name="publish_state_frequency" value="2.0"/>

  <!-- These transformations are required when the frame of the robot fleet is
  different from that of RMF globally. In order to transform a pose from the RMF
  frame to the free fleet robot frame, it is first scaled, rotated, then
  translated using these parameters -->
  <param name="scale" value="0.928"/>
  <param name="rotation" value="-0.013"/>
  <param name="translation_x" value="-4.117"/>
  <param name="translation_y" value="27.26"/>

</node>
```

Furthermore, an example of this configuration can be found in the repository as well, under the packages `ff_examples_ros1` and `ff_examples_ros2`. This example launches the example